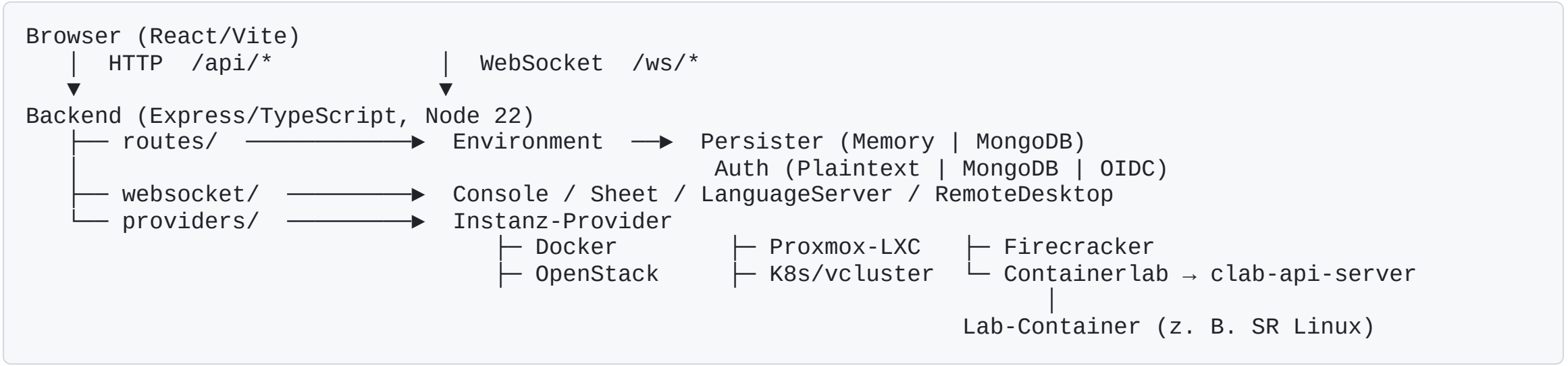
learn-sdn-hub — Code-Tour

Bachelor-Projekt AI1021 · WiSe 2026/27 · NetLab
Stand: `master` = `88a8b7c` (06.10.2026)

Was ist learn-sdn-hub?

- Web-basierte Lehr-/Lernplattform für Netz- und IT-Labore (SDN/P4, Cloud, Programmierung).
- Studierende bekommen **im Browser**: Lab-Sheet, Terminal(s), Datei-Editor (+ optional Remote-Desktop, Web-Frames).
- Aufgaben („Assignments") werden als **isolierte Instanzen** deployt: Docker-Container, Proxmox-LXC, Firecracker-microVM, OpenStack-VM, Kubernetes-Pod oder **containerlab-Lab**.
- Prüf-Schritte und Abgaben sind Teil des Modells → nutzbar für Übungen, Bonus und Prüfungen.
- Open Source, genutzt an h_da und HS Fulda; Paper auf der NOMS 2024.

Big Picture



Ein Assignment = Konfiguration + Lab-Sheet + Instanz-Provider + Prüf-Schritte.

Repo-Layout

Pfad	Inhalt
`backend/src/`	Express-Backend (CommonJS, ts-node, jest)
`backend/src/Configuration.ts`	**Assignment-Definitionen** (Datenmodell)
`backend/src/Environment.ts`	Kern: Deploy/Stop, Konsolen, Prüf-Schritte, Submission, Exam
`backend/src/providers/`	Provider + `ClabApiClient` + `TopologyFiles`
`frontend/src/`	React 18 + Vite + MUI + Monaco + xterm.js + yjs
`examples/`	Beispiel-Assignments, Startskripte, Sample-Topologien
`docs/superpowers/`	Spec/Plan-Artefakte für größere Änderungen
`dev-stack/k8s/`, `helm/`	Kubernetes-Deployment
`.github/workflows/`	CI

Station 1 — Einstieg ins Backend

Datei	Aufgabe
`backend/src/Server.ts`	HTTP-/WebSocket-Server starten
`backend/src/Api.ts`	Router zusammenbauen (Routen + Middleware + Provider)
`backend/src/Config.ts`	Konfiguration aus Env (z. B. OIDC-Provider)
`backend/src/routes/`	`user`, `admin`, `environment`, `k8s`, …
`backend/src/*Application.ts`	**Startpunkte je Provider** (z. B. `DockerApplication.ts`, `ContainerLabApplication.ts`)

Diese Kette endet immer bei: **Provider-Instanz + Persister + Auth-Providern**.

Station 2 — Assignment-Konfiguration (`Configuration.ts`)

Ein Assignment ist ein Objekt `EnvironmentDescription` in einer Map:

```
environments.set("l2_learning_switch", {
  type: "normal",
  terminals: [[{ type: "Shell", name: "bash", cwd: "/home/p4", executable: "make", params: [""], provideTty: true }]],
  editableFiles: [{ absFilePath: "/home/p4/…/basic.p4", alias: "basic.p4" }],
  stopCommands: [],
  steps: [{ name: "1", label: "make h1 ping h2 work", tests: [ /* siehe nächste Folie */ ] }],
  description: "…",
  assignmentLabSheet: "../assignments/p4basic.md",
});
```

Wichtige Felder: `topologyUrl` (containerlab), `providerImage` / `providerDockerCmd`, `useCollaboration`, `useLanguageClient`, `maxBonusPoints`, `isExam` / `durationMinutes`.

Station 3 — Prüf-Schritte: das Interface Übung ↔ Automatisierung

```
// Ausführung auf der Instanz, Ausgabe wird gematcht:
{ type: "SSHCommand", command: "uname -a", stdOutMatch: "(.*)Linux(.*)",
  successMessage: "Instanz erreichbar.", errorHint: "SSH-Konsole prüfen.",
  gradualAssistance: [{ failedCounter: 2, hint: "Provider-Status im Environment-View prüfen." }] }

// Suche im Terminal-Puffer (z. B. "gib whoami ein"):
{ type: "TerminalBufferSearch", terminal: "(.*)", match: "(.*)whoami(.*)",
  successMessage: "whoami gefunden.", errorHint: "`whoami` im Terminal ausführen." }
```

- `steps[].tests[]` prüft **beobachtbares Verhalten**, keine Implementierung.
- `bonusPoints` je Schritt, `maxBonusPoints` je Assignment → Bewertung/Prüfung.
- `gradualAssistance` gibt gestufte Hinweise nach Fehlversuchen.

Station 4 — `Environment.ts` (der Kern)

- Deploy/Stop über den Provider, Verwaltung **aktiver Umgebungen** je Gruppe.
- Konsolen (SSH, DockerShell über clab-api-server, Remote-Desktop, Web-Frame).
- Editierbare Dateien: Lesen/Schreiben, optionales **Collaboration-Doc** (yjs).
- Ausführung der Prüf-Schritte, Auswertung, Hinweise.
- **Submission**: Pack-/Cleanup-Kommandos, eingesammelte Dateien.
- **Exam**: Startzeit, Dauer, Deadline-Logik.

> [!NOTE]
> **Randbemerkung:**
> Sprache: `English` (Doku/Kommentare), Team-Kommunikation nach Absprache.

Station 5 — Provider-Naht (der wichtigste Erweiterungspunkt)

```
export interface InstanceProvider {
  createServer(username, groupNumber, environment, options?): Promise<VMEndpoint>;
  getServer(instance: string): Promise<VMEndpoint>;
  deleteServer(instance: string): Promise<void>;
}
export interface VMEndpoint { instance; providerInstanceStatus; IPAddress; SSHPort;
                              SSHJumpHost?; LanguageServerPort; RemoteDesktopPort? }
```

Implementierungen in `master`: `DockerProvider`, `ProxmoxProvider`, `OpenStackProvider`, `FirecrackerProvider`, `LocalMultiuserVMProvider`, `K8sProvider`, **`ContainerlabProvider`**.

> [!NOTE]
> → **Ziel:** Modulatität → Neues Backend/Umfeld andocken = dieses Interface erfüllen, kein Umbau.
>
> Wir nutzen primär: für lokale Tests `DockerProvider`, für Test/Prod `ContainerlabProvider` und später (komplexer) `K8sProvider`

Station 6 — Containerlab-Pfad (neu in `master`)

- Deployment über **clab-api-server ≥ v0.7.0** (REST + WebSocket, Bearer-JWT) auf einem containerlab-Host (auch `localhost`), gestartet per `containerlab tools api-server` oder Container-Image.
- `ClabApiClient` kapselt Auth/REST/WS; `TopologyFiles` lädt relativ referenzierte Dateien einer Topologie nach.
- Terminals: **`DockerShell`** (`{type, name, containerName}`) über die Terminal-Sessions des API-Servers — kein `sshd` in den Lab-Images nötig; alternativ `Shell` per SSH auf den Jumphost-Container.
- Variablen: `CLAB_USERNAME`, `CLAB_PASSWORD`, `CLAB_APIURL`, `CLAB_MAX_INSTANCE_LIFETIME_MINUTES`, optional `CLAB_TOKEN_DURATION_IN_MINUTES`, `CLAB_LAB_PREFIX`, `CLAB_API_TLS_INSECURE`.
- Lab-Namensschema: `${CLAB_LAB_PREFIX}${environmentId}-${groupNumber}-${username}`.

> [!NOTE]
> **Ausblick: Skalierbares Production Setup**
>
> - Für production am besten skalierbar - "state of the art" DevOps/GitOps: K8sProvider (https://github.com/prona-p4-learning-platform/learn-sdn-hub/tree/master/dev-stack/k8s)

Station 7 — Terminalpfad (Browser → Instanz)



Terminalinhalt ist im Backend zwischengespeichert (Reload, `TerminalBufferSearch`),
damit Prüf-Schritte den Terminalverlauf auswerten können.

Station 8 — Editor- und Lab-Sheet-Pfad

```
frontend: FileEditor.tsx (Monaco)          ➔  /api/…  Datei lesen/schreiben
                                    └─ optional yjs (Collaboration) + monaco-languageclient
backend:  filehandler/SSHFileHandler.ts    ➔  Datei auf der Instanz per SSH
          websocket/LanguageServerHandler.ts ➔  LSP-Proxy (Port auf der Instanz)
          websocket/SheetHandler.ts        ➔  Lab-Sheet (Markdown + Mermaid)
```

- Lab-Sheets liegen in `backend/src/assignments/*.md` (oder auf der Instanz/in der DB).
- `useCollaboration` / `useLanguageClient` sind optionale Komfortpfade im Assignment.

> [!NOTE]
> monaco und xterm.js haben das größte Code-Gewicht (viel Code - viele Deps) im Frontend!

Station 9 — Frontend-Struktur

Bereich	Inhalt
`src/Routings.tsx`	Routen: Login, `/assignments`, `/environment/:name`, `/settings`, `/admin`
`src/views/`	`Assignments`, `Environment`, `Administration`, `Login`, `Settings`
`src/components/`	`FileEditor`, `XTerm`, `SubmissionOverview`, `ActiveEnvironmentsOverview`, `assignments/*`
`src/api/`	`Request.ts` (ofetch + **zod**-Validierung), `Config.ts`, `WebSocket.ts`
`src/stores/`	zustand: `authStore`, `optionsStore` (+ Persistenz)
`src/utilities/`	`ListCompareHelper`, `TerminalObserver`

Frontend ist React 18 + Vite + MUI; kein Test-Setup vorhanden (Aufgabe: `tasks/frontend-tests.md`).

Station 10 — Qualitätsgates und CI

```
# Backend (läuft auch in der CI):
cd backend && npm test            # jest unit tests
cd backend && npm run lint
cd backend && npm run type-check

# Frontend (läuft auch in der CI):
cd frontend && npm run lint
cd frontend && npm run build

# lokal zusätzlich sinnvoll:
cd backend && npm run test:integration
```

- `build-push-images.yaml`: Lint/Type-Check/Tests + **Container-Image-Builds** (nur `master`, `feature/**`, `fix/**`, Tags).
- Geplant: `ci-checks.yaml` — dieselben Prüfungen **ohne** Image-Builds, zusätzlich für `develop` sowie `onboarding/**` und `onboarding-german/**`.

Entwicklungsumgebung (Variante A, empfohlen)

```
npm install                                    # npm-Workspaces (backend + frontend)

# Terminal 1:
cd backend && DOCKER_IMAGE=prona/p4-container DOCKER_CMD=-s npm run start:docker

# Terminal 2:
cd frontend && npm run dev                     # Vite :3000 → Proxy /api,/ws → :3001
```

- Login im Entwicklungsbetrieb: Plaintext-Mehrbenutzer-Auth (`BACKEND_USERS` , z. B. `user1:password1`).
- Persistenz: `MONGODB_URL` (geteilte Atlas-Instanz oder lokal).
- 🔐 Zugangsdaten nur über Environment-Variablen — **niemals** in Repo/Deck/Issue.

Containerlab-Variante (Variante B)

```
# auf dem containerlab-Host:
containerlab tools api-server                # ≥ v0.7.0 erforderlich

# Backend:
cd backend
export CLAB_USERNAME=… CLAB_PASSWORD=… CLAB_APIURL=https://<host>:8090
export CLAB_MAX_INSTANCE_LIFETIME_MINUTES=120
export CLAB_LAB_PREFIX="teamX-"              # Pflicht bei geteiltem Server!
export MONGODB_URL=…                         # ContainerLabApplication bricht sonst ab
npm run start:containerlab
```

Beispiel-Assignment: `Containerlab-Dnsmasq-TopologyUrl` in `Configuration.ts`,
Vorlagen unter `examples/containerlab-provider/`.

> [!NOTE]
>
> K8sProvider für production schauen wir uns im Laufe des Semesters zusammen an!

Fallstricke, die man einmal gesehen haben muss

- `CLAB_LAB_PREFIX` **leer + geteilter clab-api-server** → der Prune hält fremde Labs für eigene und löscht sie nach Ablauf der Lebensdauer.
- `start:containerlab` **ohne** `MONGODB_URL` → Abbruch beim Start.
- `start:localvm` ist ein verwaistes Skript (`LocalVMApplication.ts` fehlt).
- gNMI auf SR Linux ist **TLS-only** (57400, selbstsignierte Zertifikate → skip-verify in Testumgebungen).
- Re-Provisioning eines bestehenden Lab-Workdirs: `clab deploy --reconfigure`.
- Ein Twin als eigenes clab-Lab muss beim Teardown **mit** entfernt werden, sonst bleibt das Management-Subnetz belegt.

> [!NOTE]
> Fixt das Housekeeping 🧹 (alias: ich™) teilweise dieses Semester 😄

Wie wir zusammenarbeiten

- **Branches:** `feature/<issue>-…`, `fix/<issue>-…`; ein Ziel pro PR.
- **Erst Plan, dann Code:** kurzer Plan-Absatz (klein) oder Spec + Task-Plan (groß, Vorbild `docs/superpowers/`).
- **Review:** eine fremde Person reviewt und kann die Änderung erklären.
- **DoD:** CI grün, Doku/Beispiel aktualisiert, keine Rückstände, KI-Notiz im PR.
- **KI-Notiz:** Tool/Kategorie · Anwendungsfall (Plan, Code, Debugging, Doku) · Grad der manuellen Nacharbeit.
- **Wochenprotokoll:** Team Lead notiert, wer diese Woche was macht.

Dein Einstieg: E1 „Onboarding-Tour"

Baue dein eigenes, minimales Assignment in `learn-sdn-hub`:

1. **ein** Terminal (`Shell`, Docker-Variante) bzw. `DockerShell` (containerlab-Variante),
2. **eine** editierbare Datei,
3. **zwei** Prüf-Schritte (`SSHCommand` + `TerminalBufferSearch`),
4. Lab-Sheet in `backend/src/assignments/onboarding-tour-german.md`,
5. Registrierung in `Configuration.ts`.

Verifikation: Assignment deployen → beide Schritte grün → stoppen; `npm test`, `lint`, `type-check` grün.

Bewusst **nicht** enthalten, zu komplex am Anfang ⚠️: Collaboration, Remote-Desktop, Web-Frames, Submissions, LSP.

Wegweiser nach dem Einstieg

Pfad	Erste Aufgabe
Plattform/Kern	Provider-Vertrag + Tests (`MockProvider`), containerlab-Fehlerfälle, CI/Doku
Anwendung auf dem Zwilling	Szenarien, Live-vs-Twin-Vergleich, Intent-basierte Änderungen
Freies Werkzeug (Contract)	`twin-diff` -CLI oder UI-Widget gegen gNMIc/gNMI, andockbar über den Provider-Seam

Themen entstehen aus: offenen Issues (`#283` , `#284` , `#175` , Exam `#312–#314`), dem NDT-Strang und euren eigenen Vorschlägen.

> **Welches Team wollt ihr bilden? Woran wollt ihr arbeiten?**
> Ich arbeite selbst während des Semesters am Repo mit! Housekeeping 🧹... so werden wir auch merken was passiert wenn mehrere Leute am gleichen Repo arbeiten. Hat letztes Jahr auch gut geklappt - sehr gute Ergebnisse (K8s prodiver und (!) setup, exam environment, containerlab provider Grundgerüst) aber natürlich gab es auch Schmerzen

Referenzen

- `README.md` — Betrieb, Provider, Assignment-Konfiguration, Collaboration, OIDC.
- `backend/src/Configuration.ts` — alle Beispiel-Assignments.
- `examples/` — Beispiel-Konfiguration, Assignments, Startskripte, containerlab-Samples.
- `docs/superpowers/` — Spec/Plan-Vorbild (Containerlab-Fertigstellung).
- `onboarding-german/runbook-setup.md` — Setup im Detail · `onboarding-german/tasks/` — vorbereitete Aufgaben.

Fragen: immer als Issue im Repo stellen — die Antwort hilft auch der nächsten Gruppe.